Comparing Controller Synthesis Methods with Deadline-Miss Awareness

Melanie Gallant*, Marc Seidel*, Paolo Pazzaglia, Claudio Mandrioli,
Christoph Mark, Kevin Schmidt, Frank Allgöwer, Martina Maggio

***Abstract*—This paper presents a comparative study of real-time controller design methods for dynamical control systems subject to deadline overruns. Numerous techniques have been proposed to handle missed deadlines, i.e., including models or information of possible deadline misses directly in the control design. These approaches substantially differ in their assumptions, supported execution semantics, required runtime information, and provided guarantees, making direct comparison challenging. We address this gap with a qualitative assessment of representative deadline-miss-aware control methods. We analyze the design space of such controllers and identify the critical assumptions on the real-time constraints, the control design criteria and implementation aspects. We expose how these different assumptions lead to the different trade-offs in applicability, and control performance and guarantees. We contribute a comparison tool enabling researchers to systematically benchmark new techniques against the state of the art. The study is supported by simulations on two case studies: a Furuta pendulum and an electric motor to illustrate the practical behavior of the different approaches under missed-deadline conditions.**

I. Introduction

Control systems shape our lives in many different domains: from transportation to home appliances. These systems typically have to operate in real-time, providing answers and interacting with physical environments. Industrial studies report that many of these systems experience violations of the assumptions on the timing of the software execution, i.e., occasional deadline misses [1] that sometimes could be tolerated, at least if sufficiently rare [2]. They may result from transient overloads, execution-time variability, or communication delays [3], and, as they may deteriorate performance or affect safety, should be a fundamental design concern.

Disregarding this consideration, most control applications are still developed under the assumption that functional and timing aspects can be addressed separately. This perspective underlies abstractions such as the classic hard-deadline contract, in which missing a deadline is considered a system failure. Consequently, functional correctness is reduced to ensuring that all tasks remain schedulable. This abstraction has enabled clean design processes, but in practice seems to be overly conservative. Consequences are either over-provisioning computational resources to improve response times, or selecting controller periods large enough to guarantee deadline satisfaction in all conditions. The former carries an obvious implementation cost, whereas the latter can significantly degrade control quality. Larger sampling periods lead to worse disturbance rejection, worse performance, and reduced stability margins [4]. Controllers with shorter periods, that tolerate occasional deadline misses, achieve a better trade-off between implementation cost and control quality [5].

From a control perspective, timing effects constitute a major source of uncertainty, alongside plant disturbances and modeling errors. In software-based control systems executed on shared real-time platforms, these effects arise naturally from inherent execution-time variability, interference from co-running tasks, access to data in memories and scheduling decisions. The resulting delays are typically time-varying and may exceed one sampling interval [6]. Additional design choices are then required to handle overlapping job instances, as well as delayed or missed control updates [4].

These issues have motivated a broad body of work on controller synthesis and strategies that explicitly account for deadline overruns, by designing a specific control law that either is more robust to this type of disturbances or changes depending on the history of past deadline outcomes. However, the available techniques differ substantially in their assumptions, objectives, and intended operating conditions. They may rely on different models of timing uncertainty, actuator and scheduling semantics, and levels of runtime information. As a result, it is often difficult to understand how methods relate to one another, in which scenarios they are applicable, and what trade-offs impose in terms of guarantees, complexity, and practical deployability. This lack of a common comparison basis makes it harder both for practitioners to choose an appropriate method and for researchers to position new contributions against prior work.

In this paper, we address this gap and provide a qual-

* The first two authors contributed equally to this work. M. Gallant (*melanie.gallant@de.bosch.com*) is with Robert Bosch GmbH, Corporate Research, Germany and Saarland University, Germany. M. Seidel (*seidel@ist.uni-stuttgart.de*) is with University of Stuttgart, Institute for Systems Theory and Automatic Control, Germany. P. Pazzaglia (*paolo.pazzaglia@de.bosch.com*), C. Mark (*christoph.mark@de.bosch.com*) and K. Schmidt (*kevin.schmidt4@de.bosch.com*) are with Robert Bosch GmbH, Corporate Research, Germany. C. Mandrioli (*claudio.mandrioli@uni.lu*) is with University of Luxembourg, Luxembourg. F. Allgöwer (*allgower@ist.uni-stuttgart.de*) is with University of Stuttgart, Institute for Systems Theory and Automatic Control, Germany. M. Maggio (*maggio@cs.uni-saarland.de*) is with Saarland University, Germany and Lund University, Sweden.

This work was supported by: the German Federal Ministry for Economic Affairs and Climate Action under Grant 13IPC021 (IPCEI-CIS), the European Union's Horizon Europe program under Grant Agreement No. 101148870 (ConTestCPS), the EU INTERREG North Sea project STORM_SAFE of the European Regional Development Fund, and the "Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation" via the NEST projects *Intelligent Cloud Robotics for Real-Time Manipulation at Scale* and *DYNACON: DYNamic Attack detection and mitigation for seCure AutONomy*.

Manuscript received March 30, 2026; revised June 17, 2026; accepted July 17, 2026.

itative comparison of deadline-miss-aware controller design techniques. We identified eight methods [5], [7]–[13] that, to the best of our knowledge, represent the current snapshot of state of the art that explicitly account for deadline misses in the controller design. Each approach is based on different assumptions on (i) the real-time system, (ii) control design criteria, and (iii) implementation choices. Such variety leads to different trade-offs in applicability, and control performance and guarantees. Our goal is to identify how the different assumptions of the design approaches generate these trade-offs, and thus to their design space. While there is not one best design approach overall, with our work we offer valuable directions to researchers and practitioners, enabling them to choose the right approach for their given assumptions and required guarantees, as well as identifying gaps for future research directions. To complement the qualitative analysis, we propose a comparison tool that allows to consistently integrate and evaluate new techniques against the literature. We illustrate its usage by providing simulations based on two systems subject to deadline overrun conditions: a Furuta pendulum, a widely used benchmark for stress-testing control designs [14], and an electric motor as an industrially-relevant use-case [3]. We show simulation results of all deadline-miss-aware controllers, as well as specific examples that underline the insights from the qualitative analysis. We evaluate the control performance of different designs under varying deadline-miss probabilities and under partial state measurements. Our results expose the approaches' diverse trade-offs and targeted applications, showing the design decisions that define the targeted context, providing guidance for future research.

II. Background and basic notions

This section introduces concepts and notation that we will use in the rest of the paper. Specifically, Sec. II-A introduces the system model, Sec. II-B describes how the system may react to deadline overruns, Sec. II-C provides an overview of how to model patterns of deadline overruns, and Sec. II-D gives some background on controller types.

A. System model

We define the physical plant to be controlled. We make use of both a continuous-time nonlinear model

$$\mathcal{P} \coloneqq \begin{cases} \dot{x}(t) = f\left(x(t), u(t)\right) \\ y(t) = g\left(x(t), u(t)\right) \end{cases}, \tag{1}$$

where $f(\cdot,\cdot)$ and $g(\cdot,\cdot)$ are given functions that describe the plant dynamics, and its Linear Time-Invariant (LTI) counterpart, i.e., the model that is obtained linearizing Eq. (1) around an operating point [15],

$$\mathcal{P}_{\mathrm{LTI\,ct}} \coloneqq \begin{cases} \dot{x}(t) = A\,x(t) + B\,u(t) \\ y(t) = C\,x(t) + D\,u(t) \end{cases}. \tag{2}$$

In both equations, $x(t) \in \mathbb{R}^{n_\mathrm{x}}$ is the system state at time t, $u(t) \in \mathbb{R}^{n_\mathrm{u}}$ is the system input, and $y(t) \in \mathbb{R}^{n_\mathrm{y}}$ is the system output, with n_x, n_u, and n_y respective vector dimensions. The matrices A, B, C, and D have appropriate size and describe the evolution of the linearized system in continuous-time. We assume $t \geq 0$, $x(0) \coloneqq x_0$ and $u(0) \coloneqq u_0$.

The system input is calculated by a controller that acts periodically every T seconds and applied using a zero-order-hold policy (i.e., the control signal is kept constant during the control period). The controller is designed to guarantee stability (i.e., convergence of the system state) and maximize performance. The performance is usually expressed as a *cost function*, which captures objectives such as convergence speed and control effort. The controller is implemented as a real-time task, running on a platform that is possibly shared with other applications. An execution of the controller task, called *job*, should be completed within its corresponding real-time *deadline*. When this happens, we call it a *deadline hit*, while an execution not completing within the deadline is referred to as a *deadline overrun* or *deadline miss*.

Under ideal conditions (i.e., with no deadline overruns), the controller input of the k-th iteration of the controller task is sampled periodically at $kT, k \in \mathbb{N}_0$. Denoting with $x[k]$ the value of the plant state at time kT, i.e., $x[k] \coloneqq x(kT)$, we obtain the discrete-time counterpart of Eq. (2) as

$$\mathcal{P}_{\mathrm{LTI\,dt}} \coloneqq \begin{cases} x[k+1] = \Phi\,x[k] + \Gamma\,u[k] \\ y[k] = C\,x[k] + D\,u[k] \end{cases}, \tag{3}$$

where the matrices Φ and Γ are defined as [6]

$$\Phi \equiv \Phi(T) = e^{AT}, \quad \Gamma \equiv \Gamma(T) = \int_0^T e^{A\nu}\mathrm{d}\nu\, B. \tag{4}$$

We compare controllers compatible with the Logical Execution Time (LET) paradigm [16], i.e., where the control task reads its input $x[k]$ at the start of each period and applies the calculated control signal (i.e., the plant input) at the end of each period. In control terms, this corresponds to the design of a one-step delay controller. The system that incorporates a one-step delay controller is usually formulated using the augmented state $\tilde{x}[k] = [x[k]^\top u[k]^\top]^\top$ with size $n_\mathrm{xu} = n_\mathrm{x} + n_\mathrm{u}$.

In this paper, we compare controllers that implement or adapt a *control policies* κ. The information used by the control policy creates a distinction between different classes of controllers, which we discuss in Sec. II-D. In general, the result of the execution of the control policy $\kappa[k]$ at time step k is the control signal $u[k+1]$. However, deadline overruns may prevent the policy from providing the next value of u, as described in the following.

B. Modeling the effect of deadline overruns

Under ideal conditions, the execution of $\kappa[k]$ reads its input, e.g., $x[k]$, at kT and produces its output $u[k+1]$ within time $(k+1)T$. From the real-time perspective, this is guaranteed when the control period T is selected to ensure that $T \geq R_\mathrm{max}$ where R_max is the control task's Worst-Case Response Time (WCRT). However, selecting $T \geq R_\mathrm{max}$ often leads to controllers that are too slow in reacting to changes in the environment [4]. In fact, the selection of T is constrained by physical considerations, and the designer has to accept that the control task may occasionally overrun [1], [3], causing a deadline miss. When a job misses its deadline, the operating and runtime system can deal with its remaining

execution in different ways, i.e., they can use different *overrun strategies* [2]. The most common are:

(i) KILL: The job that overruns the deadline is aborted at its deadline, all the calculations performed up to the deadline are discarded, and the internal state of the controller is reset to the one it had at its activation. The following job is activated normally.
(ii) SKIPNEXT: The job that overruns the deadline is executed until its computation is completed. Any further job that, under nominal conditions, would have been activated while the computation is ongoing is discarded. A new job is activated only at the next release instant after the completion of the overrunning job.
(iii) QUEUE(i): The job that overruns the deadline is executed until its computation is completed. Any job that, under nominal conditions, would have been activated while the computation is ongoing is placed in a queue and started (in FIFO order) immediately after completion of the overrunning job. The queue only keeps the most recent i jobs. QUEUE(1) is the most common case [3], [5] (will also be referred hereafter as QUEUE for compactness).

In this paper, we consider the three standard overrun strategies, formally grouped in the set $\mathcal{S}_\mathrm{o} \coloneqq \{\text{KILL}, \text{SKIPNEXT}, \text{QUEUE}\}$, and we define the function $\mu[k] : \mathbb{N}_0 \to \{\mathrm{H}, \mathrm{R}, \mathrm{M}\}$ which encodes the *outcome* of the job execution of the control task at each time step $k \in \mathbb{N}_0$. Having $\mu[k] = \mathrm{H}$ ("hit") implies that the job either activated or enqueued at time kT meets its deadline, which is set at time $(k+1)T$. Having $\mu[k] = \mathrm{R}$ ("recovery") implies that a job activated at time $(k-d)T < kT$, for some $d > 0$, terminates its computation in the interval $(kT, (k+1)T]$. For periods where no new control signal is generated, i.e., with neither a timely nor a late termination of a control job, the outcome of the period is denoted with $\mu[k] = \mathrm{M}$, implying a deadline miss. Under SKIPNEXT and QUEUE(1), all three outcomes are possible, while KILL allows only H and M, since any job missing its deadline is discarded and no late termination may occur.

When a deadline miss of a control task occurs, the system does not have a new control signal $u[k+1]$ to actuate, and there are various alternatives for how to decide what value is assigned to the actuators. We refer to this decision as *actuator strategy*. Common choices in the literature [2], [17] are HOLD and ZERO. The former implies that the input applied to the system remains constant, i.e., $u[k+1] \coloneqq u[k]$. The latter implies that $u[k+1] \coloneqq 0_{n_\mathrm{u}}$. These enforcements are possibly repeated across multiple periods until a new control signal is calculated. Simulations and experimental results in literature [17], [18] show that the actuator strategy may have a significant impact on the control performance and the best choice depends on the plant dynamics. We refer hereafter to the set of actuator strategies as $\mathcal{S}_\mathrm{a} \coloneqq \{\text{HOLD}, \text{ZERO}\}$.

The value of $u[k+1]$, applied starting at time $(k+1)T$, depends on the freshest completed job and on the chosen actuator strategy $s_\mathrm{a} \in \mathcal{S}_\mathrm{a}$. Additionally, based on the chosen overrun strategy $s_\mathrm{o} \in \mathcal{S}_\mathrm{o}$, such job might have been activated more than one period before. The coupling of the actuator input and the control signal can be formally defined as

$$u[k+1] = \begin{cases} \kappa[k] & \mu[k] = \mathrm{H} \\ u[k] & \mu[k] = \mathrm{M} \wedge s_\mathrm{a} = \text{HOLD} \\ 0_{n_\mathrm{u}} & \mu[k] = \mathrm{M} \wedge s_\mathrm{a} = \text{ZERO} \\ \kappa[k-d] & (\mu[i])_{i \in \{k-d-1,\dots,k\}} \in \mathcal{R}_{s_\mathrm{o}}(d), \end{cases}$$

where, for $s_\mathrm{o} = \text{SKIPNEXT}$,

$$\mathcal{R}_{\text{SKIPNEXT}}(d) = \{(\mu_0, \dots, \mu_{d+1}) \mid \mu_0 \in \{\mathrm{R}, \mathrm{H}\} \\ \wedge\ \mu_i = \mathrm{M},\ \forall i \in \{1, \dots, d\} \wedge \mu_{d+1} = \mathrm{R}\},$$

and, for $s_\mathrm{o} = \text{QUEUE}$,

$$\mathcal{R}_{\text{QUEUE}}(d) = \{(\mu_0, \dots, \mu_{d+1}) \mid \mu_{d+1} = \mathrm{R} \wedge \\ [(\mu_0 = \mathrm{H} \wedge \mu_i = \mathrm{M}\ \forall i \in \{1, \dots, d\}) \\ \vee\ (\mu_1 = \mathrm{R} \wedge \mu_i = \mathrm{M}\ \forall i \in \{2, \dots, d\})]\}.$$

Based on the selected overrun and actuator strategy, the possible sequences of missed and hit deadlines that the control task may experience impact the closed-loop system differently, possibly affecting also the feasibility of designing a control policy that guarantees closed-loop properties such as stability.

C. Modeling the patterns of deadline overruns

Controllers that are aware of deadline overruns and actively react to these events often use knowledge about their occurrence, usually encoded as patterns of deadline hits and overruns. The models of such patterns can be classified in two categories: *probabilistic* and *deterministic*.

Probabilistic schedulability research provides methods for quantifying or bounding deadline-miss rates and deadline-miss probabilities, potentially based on WCRT analysis [19]. There are many factors that are causing overload conditions, such as variable execution times, interference from other tasks, cache misses, etc.—and their interplay is not obvious. This makes the exact computation of a probabilistic model nontrivial [20]–[22]. The related works considered in this paper use tools such as scenario theory and sampling-based statistical analysis to provide a probabilistic model of deadline hit/miss patterns. Frequently, the observed probabilistic patterns are modeled using Markov chains and their corresponding Markov graphs [12], [23] and represented as probability-based transition matrices. When employed in a Markov Jump Linear System (MJLS) framework, these models facilitate the analysis and control of systems with stochastically switching dynamics.

Deterministic models describe properties that the patterns of deadline outcomes should satisfy. The commonly used deterministic models are the (h, w)-firm[1] model [24] and Weakly-Hard (WH) constraints [25], as synthesized below:

(i) **AnyHit**(h, w) or (h, w)-firm: in any w consecutive outcomes there are at least h successful ones (H or R).
(ii) **AnyMiss**(m, w): in any w consecutive outcomes there are at most m deadline overruns (M). This model is related to the (h, w)-firm model with **AnyMiss**$(m, w) \equiv$ **AnyHit**$(w - m, w)$.

[1]In the literature the (h, w)-firm model is usually referred to as (m, k)-firm model. We renamed the variables for consistency as we use m to indicate deadline misses, and k for the discrete time step.

Directed Graph: a tuple $\langle \mathcal{V}, \mathcal{E} \rangle$ where $\mathcal{V}$ is a set of nodes and $\mathcal{E}$ is a set of labeled edges. Specifically, given an alphabet of labels Σ, we have $\mathcal{V} = \{v_1, \ldots, v_{n_\mathcal{V}}\}$ and $\mathcal{E} = \{e_1, \ldots, e_{n_\mathcal{E}}\} \subseteq \mathcal{V} \times \Sigma \times \mathcal{V}$. An edge $e_k = (i, l, j) \in \mathcal{E}$ represents a connection from node $i \in \mathcal{V}$ to node $j \in \mathcal{V}$ labeled with $l \in \Sigma$.

Markov Graph: a directed weighted graph, i.e., a tuple $\langle \mathcal{V}, \mathcal{E} \rangle$ where the label $l \in [0, 1]$ of an edge $e_k = (i, l, j) \in \mathcal{E}$ is the transition probability from Markov mode i to mode j.

Box 1: Directed and Markov graph definitions.

(iii) **RowHit**(h, w): in any w consecutive outcomes there are at least h *consecutive* successful ones (H or R).

(iv) **RowMiss**(m): in any window there are at most m *consecutive* deadline overruns (M). Originally defined with a window, this model is actually independent of the window size and **RowMiss**$(m) \equiv$ **AnyMiss**$(m, m+1)$.

Each of the deterministic models above corresponds to a *regular language*. This can be proved formally by first encoding every job outcome over the binary alphabet $\Sigma = \{0, 1\}$, where 0 is a deadline miss (M) and 1 a successful outcome (H, R). A sequence of task executions is a word $w \in \Sigma^*$, and a deterministic model λ defines the language $\mathcal{L}(\lambda) = \{w \in \Sigma^* \mid w \models \lambda\}$ of all finite outcome sequences that satisfy it. Constraints (i)–(iv) are *regular* languages because all of them are *sliding-window* properties of bounded memory: whether a word w continues to satisfy λ after appending a new outcome $c \in \Sigma$ depends only on a bounded-length suffix of w. For **AnyHit**(h, w) and equivalently **AnyMiss**(m, w), the acceptance of the next outcome is determined entirely by the last $w - 1$ outcomes, since the constraint is re-evaluated on every window of w consecutive jobs. For **RowHit**(h, w) the same bound applies, while for **RowMiss**(m) it suffices to remember the number of trailing consecutive overruns, which is capped at m and hence also finite. In every case, the set of suffixes that the automaton must distinguish is finite. By the Myhill-Nerode theorem [26], $\mathcal{L}(\lambda)$ is thus regular and recognized by a finite-state machine whose states are exactly the relevant bounded suffix of the outcome history and whose transitions are labeled by the feasible next outcomes in Σ.

Regular languages can be represented using directed graphs (see Box 1). Graph edges either correspond to a single outcome [8], [27], [28] (H, M, R), or concatenate multiple outcomes using a *lifted* model [13], [29]–[31].

D. Controller classes

The control policy $\kappa[k]$ can be classified depending on its inputs. Table I shows the classification, rooted in control theory [32], where different dependencies may be combined, as indicated using $(\cdot)$.

The first distinction partitions state- and output-feedback controllers. State-feedback controllers require to measure all system states (i.e., the vector $x[k]$ at time step k), while output feedback controllers rely on measuring the system output (i.e., the vector $y[k]$ at time step k). The second distinction splits controllers into dynamic and static. Dynamic controllers use internal information (such as estimates for physical quantities and integral of the error, stored in the controller state $z \in \mathbb{R}^{n_z}$), while static controllers do not possess an internal state.

TABLE I: Controller classes

Class	Controller input	Notation
State-feedback	System state $x[k]$	$\kappa(x, \cdot)$
Output-feedback	System output $y[k]$	$\kappa(y, \cdot)$
Dynamic	Controller state $z[k]$	$\kappa(z, \cdot)$
Static	–	$\kappa(\cdot)$
Switching	Pattern $(\mu[i])_{i \in \{k-j-1, \ldots, k-1\}}$	$\kappa(\mu, \cdot)$
Time-dependent	k	$\kappa(k, \cdot)$

A special type of internal state for switching controllers is the outcome of past iterations. Some of the controllers compared in this paper use knowledge of a sequence of past outcomes $(\mu[i])_{i \in \{k-j-1, \ldots, k-1\}}$ to adjust the controller calculations. In Table I, this sequence contains the last j outcomes, which, for patterns of deadline hits and overruns, typically correspond to the most recent window w. For controllers designed using graph-based deadline miss models, adaptation to past outcomes is usually implemented by introducing a dependency of the control policy on the graph nodes encoding the current history of μ. In the following, this is referred to as a node-dependent controller.

Finally, some controller may take time into account in the definition of the policy, for example by being more aggressive at the start and less aggressive as time progresses.

For each of the controllers analyzed in the following section, we will indicate to what class the controller belongs, and hence what set of information is needed for its implementation.

III. Deadline-miss-aware controllers

We have selected eight methods for controller synthesis or adaptation that result in control systems that are deadline-miss aware, i.e., [5], [7]–[13]. In principle, other methods could also be adapted[2] to the same setting, especially the ones dealing with packet dropouts [30] in Networked Control Systems (NCS) or hardware faults [33], but we restrict our study to the handling of deadline overruns. In this section, we analyze the different characteristics of the controllers and provide brief descriptions of each design method.

Table II reports the distinguishing features of each controller. We partition these features into three categories:

a) how the method models and handles deadline overruns;
b) controller characteristics; and
c) implementation aspects.

In Table II (a), we first classify the modeling and handling of deadline misses as stochastic () or deterministic () approaches. We then indicate the specific deadline-miss models, including if the method can be generalized by extending it to consider any regular language (see Sec. II-C). Finally, we report on the supported deadline overrun and actuator strategies (see Sec. II-B), which are key design choices that can impact both applicability and performance.

[2]Adapting original work meant to deal with packet dropouts or hardware faults would require modifying the proposed approaches with explicit design choices from our side. We encourage the original proponents of these strategies to test such adaptations with our framework of Sec. V-A.

Quadratic cost $J(x,u) = \Sigma_{k=0}^{N-1} x[k]^\top Q_\mathrm{x} x[k] + u[k]^\top Q_\mathrm{u} u[k] + x[N]^\top Q_\mathrm{N} x[N]$: weighted sum of states and control action over a finite or infinite (if $N = \infty$, $Q_\mathrm{N} = 0$) time horizon, combining state convergence and control effort, with positive definite $Q_\mathrm{u} \in \mathbb{R}^{n_\mathrm{u} \times n_\mathrm{u}}$ and positive semidefinite $Q_\mathrm{N}, Q_\mathrm{x} \in \mathbb{R}^{n_\mathrm{x} \times n_\mathrm{x}}$.

Expected quadratic cost $\mathbb{E}[J(x,u)]$: a probabilistic version of the quadratic cost averaged over the possible hit-miss patterns.

ℓ_2-performance: measures the worst-case energy amplification from an (artificial) exogenous signal $\bar{u}$, representing, e.g., disturbances, to an performance output $\bar{y} = \bar{C}x[k] + \bar{D}u[k]$, representing quantities of interest, e.g, costs. The ℓ_2-gain is the smallest γ such that $\Sigma_{k=0}^{\infty} \bar{y}[k]^\top \bar{y}[k] < \gamma^2 \Sigma_{k=0}^{\infty} \bar{u}[k]^\top \bar{u}[k]$ holds.

Box 2: Cost function definitions.

Stability is defined w.r.t. an equilibrium point of the system, typically 0.

Asymptotic stability: if the system's state starts close to 0, it stays close to 0 for all times and the state $x[k]$ converges asymptotically to 0 (i.e., $x[k] \to 0$). This is often verified using spectral radius arguments, or using Lyapunov methods.

Mean-square stability: the second moment of the state $x[k]$ converges to 0 (i.e., $\mathbb{E}\{x[k]^T x[k]\} \to 0$), indicating the stochastic convergence of $x[k]$ to 0.

Box 3: Stability definitions.

In Table II (b), we outline the choices concerning the control design. These include the selection of the control policy class (Sec. II-D), the theoretical guarantees offered by the method, and the targeted performance criteria, i.e., the cost function used in the control-design optimization problem. We outline the cost functions relevant to this paper in Box 2. The theoretical guarantees can concern either the stability (see Box 3) of the closed-loop system, or control performance metrics, including constraints on states and inputs.

In Table II (c), we consider the implementation aspects: This includes whether the control policy needs to know the deadline-miss pattern μ, the computational load (offline for the design, and online for the execution of the policy κ), and the control policy's memory footprint. The offline part is run once at design time, while the online part consists of the computations done in every control job. The memory footprint influences the storage requirements of the controller task.

In the following, the compared controller synthesis methods are briefly described.

A. `AccControl` [7]

This controller synthesis method uses an **`AnyHit`**(h,w) model under KILL and HOLD, including an additional assumption that some deadlines corresponding to a worst-case execution pattern are always met. Following this special model, *mandatory* and *optional control jobs* are defined. Jobs that must hit their deadline in order to fulfill the constraint are classified as mandatory, while all other jobs are optional. The periodicity of the mandatory jobs' pattern influences scheduling decisions.

The model allows to introduce the notion of *accelerable control* (`AccControl`): A control law is accelerable if any deadline hit of an optional job improves the control performance. The controller is then designed based on the worst-case known hit pattern, i.e., assuming that only the deadline of the mandatory jobs hit their deadlines. Using the Bellman optimality principle yields an optimization-based control design method, in which this controller is refined to make it accelerable. This method is compatible with the LET paradigm using state augmentation. While theoretically this synthesis method can work for nonlinear systems and for any general performance index, the resulting problem is computationally tractable only for certain classes of systems. For LTI systems, the work specifically designs a controller for quadratic costs, solving Algebraic Riccati Equations (AREs).

B. `GraphLQR` [8]

This control synthesis method assumes an **`AnyHit`**(h,w) deadline-miss model, and models the closed-loop system as a switched system with constrained switching. This is one of the first controller synthesis methods that introduces a graph to describe the admissible patterns of deadline hits and misses. In `GraphLQR`, state-feedback controllers are designed using a Linear-Quadratic Regulator (LQR) design method with an infinite horizon quadratic cost function. The method supports the KILL and ZERO strategies, but can be extended to HOLD.

The approach is a two-step procedure. First, a standard LQR for the case without deadline misses is analyzed with respect to its worst-case performance loss under the deadline-miss model. The analysis can be performed using Linear Matrix Inequality (LMI)-based tools, resulting in solving a Semi-Definite Program (SDP), or using a more conservative analytic upper bound. Second, a node-dependent controller is designed based on the graph. The design is based on either solving an SDP or on using a computationally less expensive (but more conservative) iterative design approach that exploits the LQR from the first step. The method guarantees asymptotic stability and an upper bound on the worst-case quadratic cost.

C. `DMAC` [5]

This controller synthesis method uses a stochastic model for deadline misses and an LTI plant model. The approach considers the reactive overrun strategies, KILL, SKIPNEXT, and QUEUE. It uses HOLD as actuation strategy, but an extension to ZERO appeared in [34]. The approach requires performing a certain number of simulations for the scheduling of the entire task set to obtain the worst-case probabilities of specific patterns of deadline overruns, and identifying the switched dynamics equations under the different strategies.

To provide robustness guarantees on the estimation of the overrun probabilities, the approach uses *scenario theory* [35]. This ensures that the most pessimistic sequence selected in a set of simulations can be classified as close to the real "worst" sequence under given robustness and confidence levels. Based on the sequence, a fixed Linear Quadratic Gaussian (LQG) controller is obtained by solving the corresponding Stochastic Riccati Equation (SRE), paired with a linear predictor. The resulting controller is shown to have better performance than a controller with larger period without overruns and SKIPNEXT is shown to obtain better performance on average with respect to the other two strategies for small control periods (i.e., in case of higher probabilities of missing deadlines).

TABLE II: Control method comparison: overview

(a) Deadline-miss model and real-time setting

Method (authors, year)		Deadline-miss model	Extendable to regular lang.	Overrun strategy s_o	Actuator strategy s_a
`AccControl` (Gaid et al.'08) [7]	⚙	**`AnyHit`**(h, w) with specific predetermined periodic hit pattern	✗	KILL	HOLD
`GraphLQR` (van Horssen et al.'16) [8]	⚙	**`AnyHit`**(h, w)	✔	KILL	ZERO, (HOLD)
`DMAC` (Pazzaglia et al.'19) [5]	◆	n.a. - based on example schedules		KILL, SKIPNEXT, QUEUE	ZERO, HOLD
`StrAdapt` (Pazzaglia et al.'21) [9]	⚙	adaptable to **`RowMiss`**(m)	✗	SKIPNEXT	HOLD
`DMAI` (Vreman et al.'22) [10]	⚙	**`RowMiss`**(m)	✗	KILL	ZERO, HOLD
`DR-MJLS` (Gallant et al.'24) [11]	◆ / ⚙	$\sim$ **`RowMiss`**(1)	✔	Hybrid strategy[a]	HOLD[a]
`TreeMPC` (Gallant et al.'25) [12]	◆	**`RowMiss`**(m)	✔[b]	KILL, SKIPNEXT, QUEUE	ZERO, HOLD
`GSS` (Seidel et al.'26) [13]	⚙	naturally: any given by a regular language[c]		KILL, SKIPNEXT	ZERO, HOLD

(b) Controller design

Method	Controller class	Performance criteria for design	Theoretical guarantees
`AccControl` [7]	$\kappa(k, x)$	general performance metric; e.g., quadratic costs for LTI systems	retains guarantees of controller designed for periodic hit pattern, more hits improve performance
`GraphLQR` [8]	$\kappa(x, \mu)$	infinite-horizon quadratic cost	asympt. stability and quadratic costs upper bound
`DMAC` [5]	$\kappa(x)$	expected quadratic cost term	none
`StrAdapt` [9]	$\kappa(y, \mu)$ or $\kappa(y, \mu, z)$	general performance metric	a-posteriori stability check based on JSR analysis
`DMAI` [10]	$\kappa(y, \mu)$ or $\kappa(y, \mu, z)$	n.a. (adapts a given controller)	a-posteriori stability and performance analysis
`DR-MJLS` [11]	$\kappa(x, \mu)$ or $\kappa(x)$	criteria depend on objective function (e.g., worst-case expected infinite-horizon quadratic cost)	mean-square stability with confidence $1 - \beta$
`TreeMPC` [12]	$\kappa(x, \mu)$	expected quadratic cost term of states and inputs and minimization of soft state constraints violation	mean-square stability, recursive feasibility, satisfaction of input/state constraints
`GSS` [13]	$\kappa(x, \mu)$ or $\kappa(x)$	ℓ_2-performance (worst-case energy amplification)	asymptotic stability and ℓ_2-performance

(c) Implementation aspects

Method	Miss-knowledge	Comput. load: Offline	Comput. load: Online[d]	Memory footprint
`AccControl` [7]	✗	w AREs (LTI systems)	$\square_{n_\mathrm{u} \times n_\mathrm{xu}} \cdot \square_{n_\mathrm{xu}}$ (LTI systems)	w time-dependent gains[e] (LTI systems)
`GraphLQR` [8]	✔	SDP or iterative SDP design	$\square_{n_\mathrm{u} \times n_\mathrm{xu}} \cdot \square_{n_\mathrm{xu}}$	node-dependent gains[e]
`DMAC` [5]	✗	SRE	$\square_{n_\mathrm{u} \times n_\mathrm{xu}} \cdot \square_{n_\mathrm{xu}}$	node-independent gain[e]
`StrAdapt` [9]	✔	$c \times$ SDP or classic tuning	$\square_{n_\mathrm{u} \times n_\mathrm{xu}} \cdot \square_{n_\mathrm{xu}}$	depends on WCRT and granularity
`DMAI` [10]	✔	simple matrix computations	$\square_{(n_\mathrm{z}+n_\mathrm{u}) \times (n_\mathrm{z}+2n_\mathrm{y})} \cdot \square_{n_\mathrm{z}+2n_\mathrm{y}}$	node-dependent dynamic controller[e]
`DR-MJLS` [11]	✔or ✗	SDP	$\square_{n_\mathrm{u} \times (n_\mathrm{x}+2n_\mathrm{u})} \cdot \square_{n_\mathrm{x}+2n_\mathrm{u}}$	node-dependent or -independent gains[e]
`TreeMPC` [12]	✔	SDP	SOCP	node-dependent OCPs
`GSS` [13]	✔or ✗	SDP	$\square_{n_\mathrm{u} \times n_\mathrm{xu}} \cdot \square_{n_\mathrm{xu}}$	node-dependent or -independent gains[e]

[a] Extendable to any overrun strategy $s_\mathrm{o} \in \mathcal{S}_\mathrm{o}$ and actuator strategy $s_\mathrm{a} \in \mathcal{S}_\mathrm{a}$. [b] Probabilities are required. [c] Includes all WH constraints.
[d] Matrix-vector multiplication is denoted as $\square \cdot \square$, with respective dimensions. Control methods with miss-knowledge require also the current node.
[e] The number of stored coefficients (per node-dependent gain) deduces from the size of the matrices in the previous column.

D. `StrAdapt` [9]

This work targets control applications where overruns may happen sporadically, and the knowledge of the timing model is limited to the WCRT R_max of the control task (and thus adaptable to the **`RowMiss`**(m) model). The underlying control design is a standard periodic one-step-delayed feedback controller with actuator handling HOLD, and with period T being either a multiple of (or equal to) the sensor period T_s. The proposed method extends this basic controller with an adaptive design based on the *continuous stream* model of computation [36]. Here, a job missing its deadline is allowed to continue executing by "stretching" its periodic instance by multiples of the sensor period until completion, while postponing the activation of the next control job. The next job then re-initializes the periodic pattern and can effectively execute without any interference from the previous job. For the case of $T = T_\mathrm{s}$, such overrun strategy coincides with SKIPNEXT. To compensate the deviation in the expected state evolution during an overrun, the job that is activated after an overrunning one is executed with a different controller, tuned as a function of the length of the overrun (quantized by the number of sensing periods T_s). Under the assumption that the WCRT is known, a finite number of control tunings is thus required, equal to $c = \lceil (R_\mathrm{max} - T)/T_\mathrm{s} \rceil$. Asymptotic stability is evaluated in terms of the Joint Spectral Radius (JSR) and the approach is compared against other non-adaptive designs under different WCRT conditions, showing increased flexibility and performance.

E. `DMAI` [10]

`DMAI` is an implementation method for dynamic LTI controllers to improve the system performance and stability under deadline misses, that does not affect the system's performance

in their absence. While the controller adaptation is evaluated only for the HOLD case, the approach is independent of the actuator strategy. However, it requires the KILL overrun strategy, and knowledge, during execution, of the number of deadlines that have been missed since the last hit (using **RowMiss**(m)). When one or more deadline misses occur, the proposed approach updates the controller state with the value that it would have had if the deadline misses did not occur. This computation is performed using the controller equations and propagating its state over time based on the number of missed task executions. The missing sensor readings of the killed jobs are approximated with a linear interpolation between the reading from last hit and the newest one. This translates in making the LTI controller matrices a function of the number of consecutive deadlines that have been missed.

The approach requires a set of six matrices for each number of possible consecutive deadline misses, which can be computed offline and stored to avoid increasing the online computational cost. The size of each matrix is upper bounded by the size n_z of the controller's state. While the paper does not provide theoretical guarantees, it includes a probabilistic analysis of the performance and a JSR-based analysis of the stability showing a significant improvement against the non-adaptive controller for many industrial plants.

F. DR-MJLS [11]

DR-MJLS is a synthesis method that finds a *distributionally robust* state feedback control method for stabilizing discrete-time *MJLSs*. A real-time control system subject to bounded probabilistic deadline overruns can be modeled as such a switching system, by encoding distinct parts of hit/miss patterns as Markov modes with uncertain transition probabilities. An example is provided, where a hybrid overrun strategy incorporating both KILL and QUEUE(1) in combination with HOLD is used. The example section briefly motivates different sources of uncertainty regarding the probabilities of missing a deadline that are compensated for in the controller design. The approach guarantees (with a given confidence) mean-square stability for MJLSs, where the transition probabilities can be time-varying and/or are estimated based on a finite number of samples. Depending on the available data, the resulting controller can be robust over the full space of probabilities (i.e., deterministic) or for a limited set of possible probabilities, converging for a large number of samples to a fully stochastic one. While the paper shows only one specific example with **RowMiss**(1), the extension to other deadline-miss models, overrun, and actuator strategies is straightforward based on the steps used to model the included example system. Either a node-dependent or a non-switching state-feedback controller is designed offline based on distributionally robust Lyapunov-type conditions transformed into sets of LMIs. The offline design scales with $\mathcal{O}(n_{\mathrm{xu}}^{6.5} \ln n_{\mathrm{xu}})$ and $\mathcal{O}(n_{\mathcal{V}}^{3.5} \ln n_{\mathcal{V}})$.

G. TreeMPC [12]

TreeMPC is a stochastic Model Predictive Control (MPC) approach for MJLSs. The method guarantees mean-square stability by optimizing a finite horizon expected quadratic cost, given the transition probabilities. Additionally, it provides robust satisfaction of any hard input/state constraints, and penalizes violations of soft constraints. The original formulation encodes **RowMiss**(m) but can be extended to any regular language, if a probabilistic modelization is available. The work demonstrates implementations for the overrun strategies KILL, SKIPNEXT, and QUEUE(1), combined with the actuator overrun models ZERO and HOLD. Based on a MJLS model, the controller is designed by spanning a scenario tree of potential future patterns of deadline outcomes from each relevant initial mode and compiling Optimal Control Problems (OCPs) using the predicted state trajectories for these trees.

To guarantee recursive feasibility of the online optimization problem, the offline design step synthesises a mode-dependent terminal set and cost function by solving a set of LMIs that scale with $\mathcal{O}(n_{\mathrm{xu}}^{6.5} \ln n_{\mathrm{xu}})$ and $\mathcal{O}(n_{\mathcal{V}}^{3.5} \ln n_{\mathcal{V}})$. The online phase of a control job involves compiling the augmented state vector using state measurements and auxiliary variables, then solving an OCP for the current initial mode. Due to the scenario-tree formulation, these OCPs can be formulated as computationally tractable Second-Order Cone Programs (SOCPs) and solved efficiently using interior point methods.

H. GSS [13]

The graph-based switched system controller design approach (GSS) is an LMI-based synthesis method that explicitly incorporates the deadline-miss model in the design phase. It considers all evolutions of the system under all possible patterns of deadline outcomes. For all the patterns, it provides rigorous guarantees for asymptotic stability and ℓ_2-performance, even in the worst-case. Analyzing a given controller for its achieved performance is also possible. This is thus a deterministic approach, that can handle ZERO and HOLD, and the two overrun strategies KILL and SKIPNEXT. The design part is done offline: A switched system description of the WH control system together with the concept of multiple Lyapunov functions is used to formulate a set of LMIs, which result in a computationally tractable SDPs. Its computational effort scales with $\mathcal{O}(n_{\mathrm{xu}}^{6.5} \ln n_{\mathrm{xu}})$ and $\mathcal{O}(n_{\mathcal{V}}^{3} n_{\mathcal{E}}^{0.5} \ln n_{\mathcal{E}} + n_{\mathcal{V}}^{2} n_{\mathcal{E}}^{1.5} \ln n_{\mathcal{E}})$. Note that the (lifted) graph used in this work is smaller than the one used in other graph-based methods.

The resulting controller additionally utilizes the last applied input, which improves performance. It is possible to either design multiple state-feedback controller gains, from which at start of each job one is picked depending on the past hit/miss-pattern (resulting in a node-dependent controller), or to design a single controller gain valid for all admissible hit/miss-patterns, although more conservative. The only requirement on the deadline-miss model is that it can be represented by a labeled directed graph, making it thus naturally applicable to any WH constraint, but also to any deadline-miss model that corresponds to a regular language.

IV. QUALITATIVE COMPARISON

Based on Table II, we qualitatively compare the considered deadline-miss-aware controllers along two main dimensions: *applicability*, and *control design criteria and guarantees*. Our objective is to expose the trade-offs that appear in the state-of-the-art controllers with deadline-miss awareness. Such

trade-offs can guide practitioners in the choice of a suitable controller for their applications based on (i) the assumptions about the system, the expected implementation effort and available computational resources, and (ii) the desired control performance and guarantees. At the same time, researchers can use this discussion to identify open research directions.

A. Applicability

Applicability mainly depends on three aspects: model assumptions on the real-time system, assumptions on the control design setting, and implementation effort offline and online.

Underlying assumptions about the real-time system. The methods differ significantly in the assumed structure for deadline hit/miss patterns. `AccControl` is the most restrictive, since it relies on a predetermined periodic pattern of mandatory hits fulfilling an **`AnyHit`**(h, w)-like constraint. This is attractive when such a pattern can be enforced, but less suitable when misses are irregular or only partially characterized. At the other extreme, `DMAI` and `StrAdapt` require only limited timing knowledge: `DMAI` needs the number of consecutive misses since the last successful execution, while `StrAdapt` requires only the task's WCRT and adapts the controller to the overrun length. These methods are therefore easier to deploy when no precise miss model is available.

A middle ground is provided by methods that explicitly encode admissible miss patterns. `GraphLQR` uses an (h, w)-firm model but can be extended to regular languages, while `GSS` handles any deadline-miss model representable as a labeled graph, including WH constraints and regular languages. `TreeMPC` and `DR-MJLS` are also flexible, since their MJLS formulation can encode different hit/miss histories and execution semantics, provided the corresponding modes and transitions are available. In practice, the closer this model is to the actual platform behavior, the less conservative the synthesis becomes.

A second applicability aspect are the supported combinations of overrun and actuator strategies. As shown in Table II (a), several methods are tied to KILL, while fewer also support SKIPNEXT. Likewise, some methods are derived only for HOLD or only for ZERO, whereas `DMAC`, `TreeMPC`, `DR-MJLS`, and `GSS` cover a broader set of combinations. This is an important distinction because the reaction to overruns directly changes the closed-loop dynamics, as discussed in Sec. II-B. Hence, a theoretically appealing method may still be unusable if the runtime semantics do not match.

Most approaches also assume availability of online information about recent deadline outcomes. This introduces integration effort and state bookkeeping. Sequence-aware controllers such as `GraphLQR`, `TreeMPC`, and `GSS` use this information to adapt gains or optimization problems to the current timing mode, reducing conservatism at the cost of higher complexity. By contrast, fixed-gain approaches such as `DMAC` reduce runtime dependence on miss-history tracking, but forfeit the added degree of freedom through adaptation.

Underlying assumptions for the control design. The second column of Table II (b) shows that the most common controller structure is sequence-aware static state-feedback. This is the case for `GraphLQR` and the node-dependent versions of `DR-MJLS` and `GSS`. Similarly, `TreeMPC` proposes a non-static sequence-aware state feedback control policy and `AccControl` also uses state feedback, but with explicit time dependence induced by the mandatory/optional job pattern. These methods are attractive when full state measurement is available or when an observer can be incorporated. However, this may be restrictive when only outputs are measured. `StrAdapt` and `DMAI` are advantageous here, since they support output-feedback and dynamic controllers and can therefore be integrated into a broader class of existing industrial designs.

Another distinction is whether a method synthesizes a controller from scratch or adapts an existing one. `GraphLQR`, `AccControl`, `DR-MJLS`, `TreeMPC`, and `GSS` are synthesis frameworks requiring a plant model and producing a dedicated control law. By contrast, `DMAI` is an implementation-level adaptation of a given controller, which is attractive when redesign is undesirable. `StrAdapt` lies in between, starting from a periodic controller and augmenting it with a finite set of retuned variants. Thus, `DMAI` and `StrAdapt` are more relevant in legacy settings to robustify existing controllers.

Finally, most methods assume an LTI plant model, or are mainly tractable in that setting. Nonlinear systems are only marginally addressed, which reflects a broader limitation of the current state of the art. While there has been some work on the analysis of nonlinear systems under WH constraints [37], [38], the controller design for those systems, even for certain system classes, is an interesting topic for future research. Note that `AccControl` proposes a design method for nonlinear plants that is based on Bellman's optimality principle, which is in general hard to solve for nonlinear dynamics.

Implementation effort. Offline, optimization-based synthesis is the dominant paradigm. `GraphLQR`, `DR-MJLS`, `GSS`, and `TreeMPC` (for terminal ingredients) rely on convex programs such as SDPs, while `TreeMPC` additionally requires the construction of scenario-trees and corresponding OCPs. This yields strong guarantees, but also a nontrivial synthesis pipeline. `DMAC` instead solves an SRE after estimating a probabilistic deadline model from scheduling simulations, shifting part of the effort to timing-data generation and calibration. `AccControl` is lighter in the LTI quadratic-cost case, where the design reduces to AREs, but this comes with stronger assumptions on the hit/miss pattern. `DMAI` has the smallest redesign burden, requiring only simple matrix computations from an existing controller, while `StrAdapt` can be implemented either through repeated tuning or by solving a bounded number of SDPs depending on the WCRT granularity.

More broadly, some methods require not only solving the control problem, but also obtaining the deadline-miss model itself. For deterministic graph-based methods, this means identifying a regular-language or WH abstraction of admissible patterns; for stochastic methods, it means estimating transition probabilities or representative schedules. In practice, this modeling step can dominate the engineering effort. Accordingly, methods with weaker timing-model requirements may be easier to adopt even if they are less specialized theoretically.

Runtime computation overhead. Most methods have light runtime load. `AccControl`, `GraphLQR`, `DMAC`, `DR-MJLS`, and `GSS` mainly reduce to selecting a gain matrix, possibly de-

pending on the current timing mode, and performing a matrix-vector multiplication. Their runtime burden is therefore modest and predictable, although sequence-aware or node-dependent controllers require storing multiple controller gains.

DMAI adds state-update computations for the adapted dynamic controller, but these remain simple linear-algebra operations. StrAdapt selects among a finite number of retuned controllers according to the observed overrun length, so its overhead scales with the discretization granularity. TreeMPC is the clear online outlier, since it solves an optimization problem during execution. Even if the resulting SOCP is tractable, this remains substantially more demanding and may limit implementation on constrained platforms. Thus, TreeMPC offers the greatest flexibility in handling constraints and stochastic evolution, but at the highest runtime computational cost.

Overall, applicability spans a spectrum. At one end are methods such as DMAI and StrAdapt, which are easier to integrate but rely on weaker timing models and more limited adaptation mechanisms. Their closed-loop behaviors depend on the baseline controllers that are not necessarily designed for good performance of the deadline-miss-aware controllers. At the other end are methods such as GSS, DR-MJLS, and TreeMPC, which explicitly encode the timing model and exploit it effectively, but require more modeling effort and, in some cases, significantly higher computational resources.

B. Control design criteria and guarantees

The second comparison dimension concerns what the methods optimize and what they can guarantee. We first discuss the design criterion and then the closed-loop guarantees.

Performance criteria for design. Criteria relating to quadratic cost terms (Box 2) dominate the literature, with AccControl, GraphLQR, DMAC, and TreeMPC all optimizing or analyzing variants of this criterion. This is natural, since quadratic objectives fit well with LTI models and lead to tractable formulations. TreeMPC extends this by combining the finite-horizon expected quadratic cost with explicit handling of soft state constraints. GSS, by contrast, uses an ℓ_2-performance objective (see Box 2), which is more aligned with robust analysis than average-cost optimality. DMAI does not define a new design criterion, since it adapts a given controller and is largely agnostic to its original objective.

Crucially, the performance criterion maps directly to the targeted operating regime, favoring nominal efficiency, robustness, or constraint handling. When average behavior under stochastic misses is of primary interest, expected quadratic cost formulations are well suited. Conversely, when miss patterns must be tolerated adversarially within a deterministic admissible set, worst-case criteria as in GSS and GraphLQR or the worst-case analysis in StrAdapt are more appropriate. In other cases, minimal implementation overhead is the priority, favoring an off-the-shelf approach like DMAI.

Stability guarantees. Most methods provide some form of stability guarantee (Box 3), depending on the framework.

For deterministic switched system formulations such as GraphLQR and GSS, the guarantee is *asymptotic stability* under all admissible switching sequences. These are strong guarantees, in that they cover all timing patterns allowed by the model. StrAdapt does not guarantee stability by design, but provides a framework for a-posteriori stability, relying on a JSR-based analysis over admissible overrun evolutions. Similarly, DMAI uses both a time-averaged performance analysis and a JSR-based analysis to analyze the performance and stability of the resulting system.

For MJLS-based stochastic approaches, the relevant notion is *mean-square stability*. TreeMPC guarantees this together with recursive feasibility. DR-MJLS guarantees mean-square stability, but in a distributionally robust sense: the guarantee holds with a certain confidence even when transition probabilities are uncertain or estimated from finite samples, making it attractive when stochastic timing behavior is not exactly known. DMAC, instead, is mainly motivated by performance under estimated stochastic miss patterns and does not provide an analogous formal stability certificate.

Taken together, these guarantees reflect the broader positioning of the methods. Graph-based deterministic methods offer strong worst-case statements under explicit admissible languages of deadline outcomes. MJLS-based methods trade such adversarial coverage for stochastic notions that better exploit probability information. Adaptation methods such as DMAI and, to a lesser extent, StrAdapt emphasize practical improvement under overruns, with lighter integration effort but generally weaker or narrower guarantees.

Additional guarantees. Only a subset of the methods provides guarantees beyond stability. GraphLQR gives an upper bound on the worst-case quadratic cost over all admissible hit/miss patterns under its (h, w)-firm model. GSS guarantees an ℓ_2-performance level in the worst case. TreeMPC is the richest in this respect: besides mean-square stability, it guarantees recursive feasibility and satisfaction of hard input/state constraints while minimizing expected cost and penalizing soft-constraint violations. AccControl provides a different but also meaningful guarantee: it preserves the guarantees of the controller designed for the mandatory hit-pattern, and any additional optional hit improves performance. By contrast, DMAC and DMAI mainly show empirical or comparative improvements without formal performance bounds. This highlights a recurring trade-off: stronger guarantees usually require more structured models and heavier synthesis, whereas lighter assumptions often come with weaker certification.

A particularly relevant differentiator is constraint handling. Among the surveyed methods, TreeMPC is the only one that supports state and input constraints in the synthesis and execution loop. This is a major advantage in safety-critical applications, where respecting bounds can be as important as convergence. The absence of explicit constraint handling does not make the other methods unusable, but they may require wrappers such as safety filters or conservative tuning.

C. Summary

No method dominates across all dimensions. Methods with broad execution-semantics support and strong guarantees typically require richer timing models and heavier offline design, whereas methods that are easier to retrofit into existing systems usually provide weaker formal certification or less expressive optimization objectives. This supports the main thesis of the

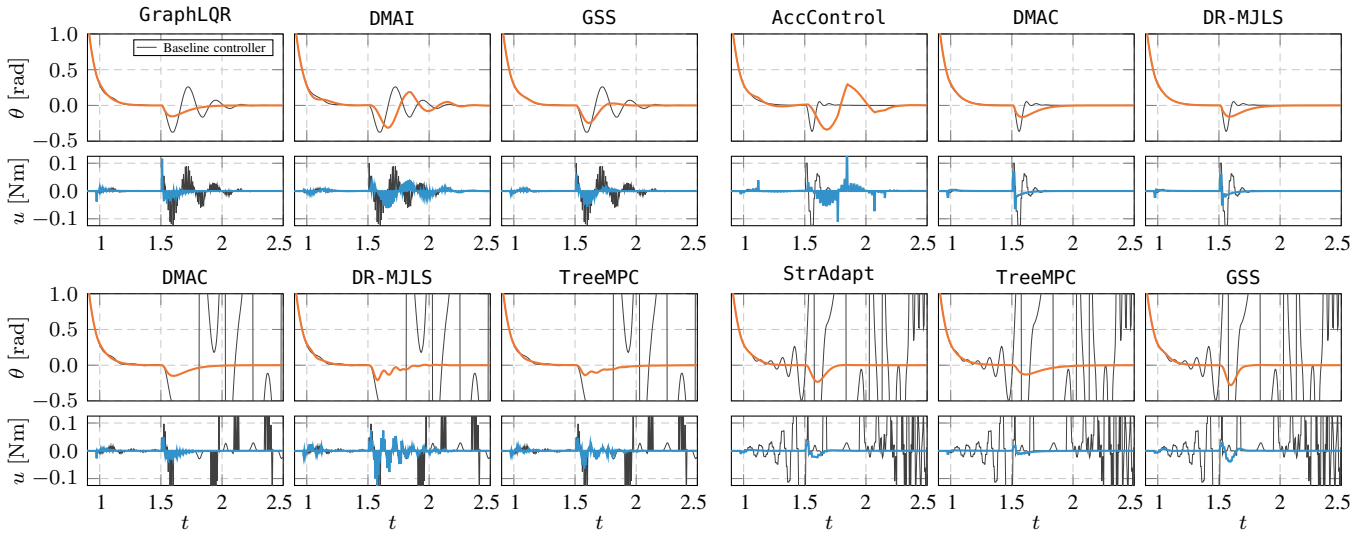


Fig. 1: Furuta pendulum: Trajectories of pendulum angle $\theta(t)$ and system input $u(t)$ with a common sequence $(\mu[k])$, using KILL/ZERO (top left), KILL/HOLD (top right), SKIPNEXT/ZERO (bottom left) or SKIPNEXT/HOLD (bottom right).

paper: deadline-miss-aware controller design is best understood as a design space with trade-offs, rather than a search for a single best solution.

V. EXPERIMENTAL RESULTS

To complement the qualitative comparison, we propose a simulation framework for quantitative scenario-specific comparisons of the controllers (Sec. V-A). To illustrate its capabilities, the framework is tested over a set of experiments on two target benchmarks, a Furuta pendulum (Sec. V-B) and an electric motor (Sec. V-C), showing examples of the effects of different design constraints and deadline-miss models. We refrain from comparing the surveyed methods in absolute terms, based only on the shown experiments: The results of the following comparisons are to be read as specific for the selected use-cases, deadline-miss scenarios, and control tuning.

A. Simulation framework

We implemented the comparison framework and all controllers in MATLAB®. The code (publicly available[3]) has been designed with a modular organization to facilitate adding and evaluating controllers in a common simulation setup.

The project comprises four subfolders. The `model` folder contains the plant and timing-constraint models and the simulation routines. Users can choose one of the provided plant examples, or add their own models and parameters. The `metrics` folder contains comparison indicators, including standard tracking and regulation measures and task-specific criteria such as maintaining the pendulum near the upright equilibrium. The `controllers` folder contains the implementations discussed in Sec. III (with optimization problems solved using the interface YALMIP [39]), and the `experimental` folder contains the scripts and configuration files for the evaluation campaigns. Together, these modules allow closed-loop simulation under selected plants, controllers, deadline hit/miss patterns, and overrun and actuation strategies, and are used to produce the results reported next.

[3]https://github.com/martinamaggio/missaware-controller-synthesis.

B. Furuta pendulum

First, we present an overview of the framework's capabilities on a comparable simulation setting for all selected methods. We use a Furuta pendulum as an academic framework, simulated using a state-of-the-art nonlinear model from [14][4] of the form in Eq. (1). For controller synthesis, the model is linearized around the upright equilibrium in the form of Eq. (2), then discretized with period $T = 5\,\mathrm{ms}$ in the form of Eq. (3). The state $x = [\theta, \dot{\theta}, \dot{\phi}]$ consists of pendulum angle θ, pendulum angular velocity $\dot{\theta}$, and base angular velocity $\dot{\phi}$, while the input u is the torque applied at the base.

We present one example simulation of all eight controllers (plus a baseline controller with no deadline-miss awareness, from [40]), with different overrun and actuator strategy combinations, subject to an offset disturbance. With the goal of fulfilling the assumptions on the deadline-miss model for all methods, a sequence of outcomes is generated that complies with the periodic hit pattern required by `AccControl` and with the corresponding **`AnyHit`**(h, w) constraint or the over-approximating **`RowMiss`**(m) constraint, where $h = 2$, $m = 3$, and $w = 5$. The Markov graph of the corresponding patterns is used for the synthesis of `DMAC`, substituting the scenario theory step. The outcomes of periods that are not required to be hits by the periodic hit model are generated probabilistically and independently, with a deadline-miss probability of $p = 0.9$.

We show all controllers at least once[5] in Fig. 1, grouped according to their respective strategy combination. The figure shows base torque $u(t)$ and pendulum angle $\theta(t)$ in the region around the upright equilibrium, where the deadline-miss-aware balancing controllers are active, after a swing-up event. The results are shown under a disturbance scenario, where an offset of $\Delta = 4\,{}^{\mathrm{rad}}/_{\mathrm{s}}$ is added to $\dot{\phi}(t)$ at $t = 1.5\,\mathrm{s}$ for two time steps.

[4]The model parameters are: pendulum arm length $0.075\,\mathrm{m}$, base radius $0.043\,\mathrm{m}$, gravity $9.81\,\mathrm{m/s^2}$, pendulum mass $0.0054\,\mathrm{kg}$ and moment of inertia $125.1 \cdot 10^{-6}\,\mathrm{kg\,m^2}$. These parameters resemble the physical pendulum used in one of the most recent synthesis papers [13].

[5]More comprehensive figures including a second disturbance scenario with brown process noise on $\theta(t)$ are available in the linked GitHub repository.

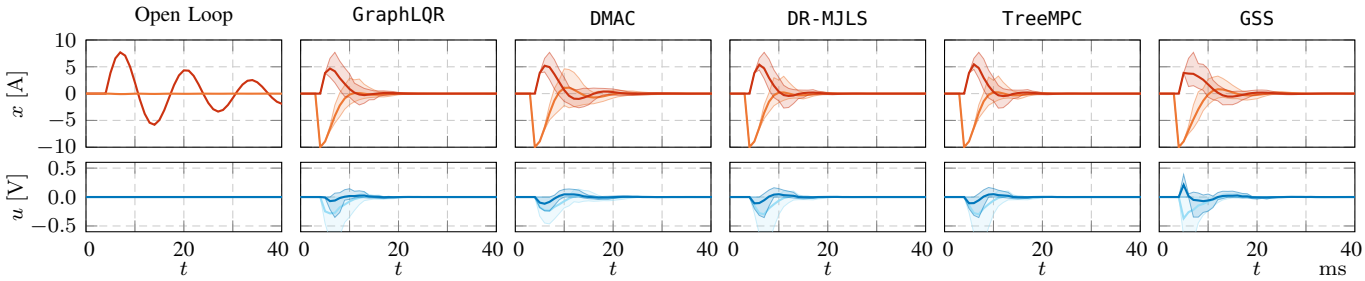


Fig. 2: Electric motor: State trajectories $i_\mathrm{d}(t)$ (—), $i_\mathrm{q}(t)$ (—) and inputs $u_\mathrm{d}(t)$ (—), $u_\mathrm{q}(t)$ (—) with offset disturbance, using KILL and ZERO. The 0.1–0.9 quantiles are shown with opacity. The trajectories of $\omega_\mathrm{el}(t)$ are omitted.

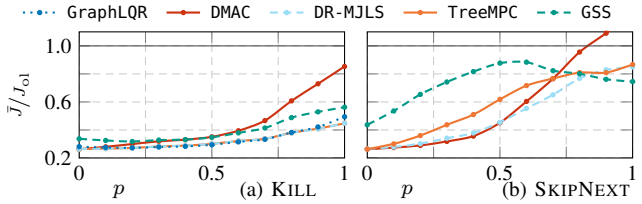


Fig. 3: Electric motor: Cost ratio $\bar{J}/J_\mathrm{ol}$ against different probabilities p of deadline misses.

The controllers with deadline-miss awareness are all stabilizing under this specific sequence of outcomes, while the system becomes unstable with the baseline controller under the SKIPNEXT strategy. However, in our tests, the controllers show different robustness. When selecting different WH constraints parameters, the methods that explicitly take into account such knowledge in the design phase to provide stability guarantees perform more consistently. On the other hand, the performance of both `StrAdapt` and `DMAI`, where stability is considered only a-posteriori, was strongly linked to the initial tuning of the nominal adapted controller. Additionally, shifting the given outcome sequence by just one or two steps can deteriorate the performance of `AccControl`. As discussed in Section IV, selecting the optimal controller is highly application-dependent and this limits the possibility of performing a universally fair quantitative comparison.

C. Electric motor

In this section, we consider a Permanent Magnet Synchronous Motor (PMSM), a common type of electric motor, as industry-relevant use-case. The PMSM is modeled[6] using the differential equations from [41] for a system state $x = [i_\mathrm{d}, i_\mathrm{q}, \omega_\mathrm{el}]^\top$, including currents in rotating *dq*-coordinates and electrical rotational speed, and input $u = [u_\mathrm{d}, u_\mathrm{q}]^\top$ comprising the applied voltages in *dq*-coordinates. The controllers are implemented with $T = 1\,\mathrm{ms}$. With our framework, multiple case studies can be defined and the available controllers compared: We consider two case studies, highlighting how different factors may affect which controller design method is the most suitable.

Influence of the deadline-miss probability. In this case study, we investigate the difference between stochastic and deterministic approaches. We focus on methods that both explicitly take into account a deadline-miss model at design time, and support a **`RowMiss`**(4) constraint with ZERO actuation strategy (HOLD is harder to implement for PMSMs due to the rotating reference frame). Whenever the WH constraint allows for a deadline miss, its probability to happen is p. We analyze how different values of p influence the control performance under different controller design methods, and if additional knowledge on those probabilities improves it. We run Monte Carlo simulations of 200 sample sequences for each value of p, extracting the quadratic costs J of each closed-loop control system, with identity matrices for $Q_\mathrm{x}, Q_\mathrm{u}$ (c.f., Box 2). Since the PMSM is open-loop stable, we compare the closed-loop performances against the open-loop quadratic cost J_ol. At $t = 3\,\mathrm{ms}$, an offset is added to ω_el, representing a sudden change in the load torque.

Fig. 2 shows the system's state and input trajectories under the selected controllers for KILL and ZERO, with $p = 0.5$: In this case, all controllers outperform the open-loop system, despite deadline misses. The averaged cost $\bar{J}$ relative to J_ol plotted over p is depicted in Fig. 3 for both KILL and SKIPNEXT overrun strategies. As expected, the costs increase with larger p, as more deadline misses typically lead to worse performance. Nonetheless, the controllers still outperform J_ol for nearly all combinations of designs and probabilities p. Furthermore, stochastic design methods like `DR-MJLS` can achieve lower costs compared to deterministic ones for some values of p. This is due to the inclusion of the knowledge on deadline-miss probabilities in their design, while deterministic designs are optimized for worst case scenarios—closer to the conditions obtained with large p. All selected controller designs use quadratic costs matching J, except for `GSS`, which optimizes ℓ_2-performance. Furthermore, `TreeMPC` has a significantly higher computation time (of one to two orders of magnitude) in this setting, however it guarantees constraint satisfaction. However, all controllers can be tuned to some extent and can behave differently for other cost functions.

Output feedback. This case study assumes that only partial state measurement ($i_\mathrm{d}(t)$ and $\omega_\mathrm{el}(t)$) is possible. Therefore, only `StrAdapt` and `DMAI` are applicable. Since both works are based on adapting a given nominal controller, we also include it in the comparison.

`DMAI` and `StrAdapt` both allow for **`RowMiss`**(m) and HOLD, but require two different overrun strategies, namely KILL and SKIPNEXT, respectively. In order to provide a comparable real-time setting, we first generate a sequence of sampled response times and derive the corresponding sequence

[6]The model parameters are: winding resistance $0.01\,\Omega$, inductance in d and q directions $1 \cdot 10^{-4}$ H and $1.2 \cdot 10^{-4}$ H, respectively, permanent magnet flux linkage $0.05\,\mathrm{Wb}$, number of pole pairs 6, and inertia $0.005\,\mathrm{kg\,m^2}$.

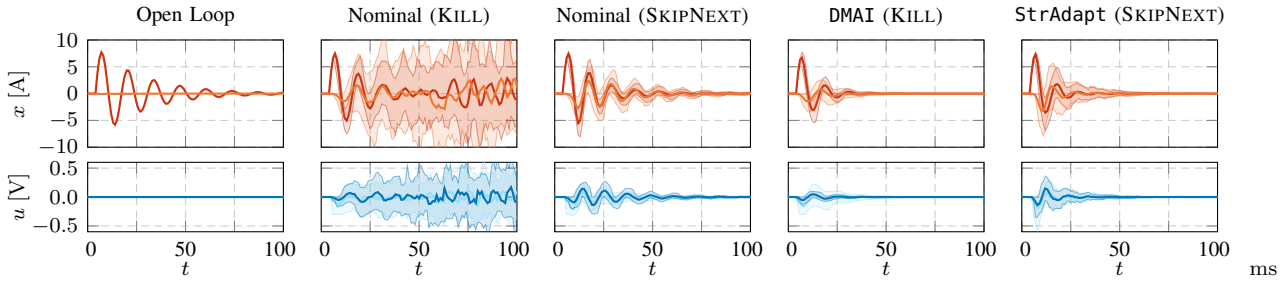


Fig. 4: Electric motor: Trajectories and 0.1–0.9 quantiles of states $i_{\mathrm{d}}(t)$ (—), $i_{\mathrm{q}}(t)$ (—) and inputs $u_{\mathrm{d}}(t)$ (—), $u_{\mathrm{q}}(t)$ (—) controlled by output-feedback at an upper bound of the job response time $\bar{t} = 2.5\,T$. The trajectories of $\omega_{\mathrm{el}}(t)$ are omitted.

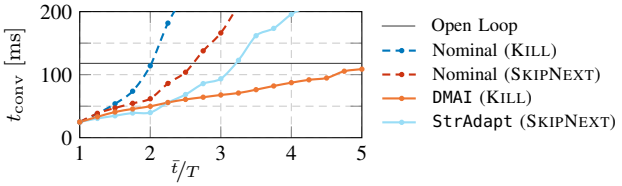


Fig. 5: Electric motor: Convergence times t_{conv} against different upper bounds of the job response times $\bar{t}$.

of outcomes for both KILL and SKIPNEXT, while ensuring a bounded value for m. Each periodic job's response time is sampled from an uniform distribution with lower bound $0.1\,T$ and parametric upper bound $\bar{t} \geq T$. We run a Monte Carlo simulation with 200 sample sequences each for increasing values of $\bar{t}$, adding the same disturbance to ω_{el} as above.

The example demonstrates that it is still possible to achieve control goals despite the absence of full state measurements and to improve performance by adapting a control policy to account for deadline misses. Fig. 4 presents the mean and quantiles of the trajectories with $\bar{t} = 2.5\,T$, for the selected controllers and overrun strategies, showing faster convergence of the deadline-miss-aware designs. Fig. 5 shows the mean convergence time for each combination, as the duration between the application of the offset disturbance and the point of time where $|x_i(t)| < 0.05\,\forall i$. Since all controllers are based on the same nominal one, they yield identical results if no deadlines are missed. However, for an increasing value of $\bar{t}$, the plots show a stronger deterioration of the nominal controllers' performance, particularly under the KILL strategy. Indeed, when sampling response times from a distribution, SKIPNEXT nonetheless ensures that the delayed job will eventually complete, while under KILL there might be a larger number of consecutive intervals without updates. Remarkably, Fig. 5 still indicates the best performance and an improvement over the open loop convergence at all points using `DMAI` and KILL, with `StrAdapt` (originally targeted at systems with sensor periods smaller than their control period) generally above, but below the nominal controllers.

VI. Related work

Controller co-design under real-time constraints has been widely studied [42], [43]. Existing works address robustness to jitter [44]–[46], exploit response-time analysis for performance improvement [47], and incorporate timing into co-simulation tools such as Jitterbug, TrueTime [4], and Jittertime [48]. Other lines of work consider the scheduling of overloaded control systems, including feedback scheduling [49], elastic tasks [50], and job skipping [51], [52]. However, these approaches mostly fall outside the scope of this paper, since they focus on schedulability while assuming pre-designed controllers.

A broad literature also studies control under network delays and losses, see [53]. The connection between deadline misses and packet dropouts is well known: under the KILL strategy, a dropped packet is equivalent to a terminated job, since the receiver has no updated command. By contrast, overrun strategies such as SKIPNEXT and QUEUE are rarely considered.

To capture such effects more expressively, several timing abstractions have been proposed, including soft deadlines [54], the (h, w)-firm model [55], and the WH model [25]. These abstractions have been used in co-design for overloaded real-time control systems and packet dropouts [7], [8], [56], [57].

More recently, the WH model has attracted growing attention due to its favorable trade-off between expressivity and modeling simplicity [58], [59]. In real-time control, deadline-miss-aware formulations were first investigated in [60], [61]. WH constraints have been first applied to packet-drop modeling in NCS in [62], later extended through graph-based representations in [29]–[31]. Controllers subject to deadline misses have been analyzed for stability requirements [3], [28], [63], [64], performance criteria [5], [18], [65], [66], safety [67], [68], security [64] and fault tolerance [66]. While a broad overview is available in [69], our paper focuses on a detailed analysis and comparison of selected deadline-miss-aware control design approaches.

VII. Conclusion

This paper provides a qualitative comparison of representative deadline-miss-aware controller design methods, together with a modular framework for their systematic evaluation. We illustrate how the considered methods differ in assumptions, runtime semantics, guarantees, and implementation effort. We also provide multiple examples of usage of the simulation framework under different deadline-miss conditions and system assumptions, underlining the qualitative results. The main takeaway is that deadline-miss-aware controller design is best understood as a trade-off space rather than a search for a universally best solution. We believe the framework and analysis presented in this paper will facilitate more reproducible comparisons and help position future contributions in this area.

Acknowledgments

We acknowledge the use of generative AI tools (Gemini 3.1 Pro) solely to assist with formatting of TikZ figures.

REFERENCES

[1] B. Akesson, M. Nasri, G. Nelissen, S. Altmeyer, and R. I. Davis, "A comprehensive survey of industry practice in real-time systems," *Real-Time Syst.*, vol. 58, no. 3, pp. 358–398, 2022.

[2] A. Cervin, "Analysis of Overrun Strategies in Periodic Control Tasks," in *Proc. 16th IFAC World Congress*, 2005, pp. 219–224.

[3] M. Maggio, A. Hamann, E. Mayer-John, and D. Ziegenbein, "Control-system stability under consecutive deadline misses constraints," in *Proc. 32nd Euromicro Conf. Real-Time Syst. (ECRTS)*, 2020, pp. 21:1–21:24.

[4] A. Cervin, D. Henriksson, B. Lincoln, J. Eker, and K.-E. Arzen, "How does control timing affect performance? Analysis and simulation of timing using Jitterbug and TrueTime," *IEEE Control Syst. Magazine*, vol. 23, no. 3, pp. 16–30, 2003.

[5] P. Pazzaglia, C. Mandrioli, M. Maggio, and A. Cervin, "DMAC: Deadline-miss-aware control," in *Proc. 31st Euromicro Conf. Real-Time Syst. (ECRTS)*, 2019, pp. 1:1–1:24.

[6] K. J. Åström and B. Wittenmark, *Computer-controlled systems: theory and design*. Courier Corporation, 2013.

[7] M. B. Gaid, D. Simon, and O. Sename, "A Design Methodology for Weakly-Hard Real-Time Control," in *Proc. 17th IFAC World Congress*, 2008, pp. 10 258–10 264.

[8] E. P. van Horssen, A. R. B. Behrouzian, D. Goswami, D. Antunes, T. Basten, and W. P. M. H. Heemels, "Performance analysis and controller improvement for linear systems with (m, k)-firm data losses," in *Proc. 2016 Eur. Control Conf. (ECC)*, 2016, pp. 2571–2577.

[9] P. Pazzaglia, A. Hamann, D. Ziegenbein, and M. Maggio, "Adaptive design of real-time control systems subject to sporadic overruns," in *Proc. Design, Automation & Test in Europe Conf. & Exhibition*, 2021.

[10] N. Vreman, C. Mandrioli, and A. Cervin, "Deadline-Miss-Adaptive Controller Implementation for Real-Time Control Systems," in *Proc. 28th IEEE Real-Time and Embedded Technology and Appl. Symp. (RTAS)*, 2022, pp. 13–26.

[11] M. Gallant, C. Mark, P. Pazzaglia, J. von Keler, L. Beermann, K. Schmidt, and M. Maggio, "Structure-Exploiting Distributionally Robust Control of Non-Homogeneous Markov Jump Linear Systems," *IEEE Control Syst. Lett.*, 2024.

[12] ——, "Soft-Constrained Stochastic MPC of Markov Jump Linear Systems: Application to Real-Time Control with Deadline Overruns," *IEEE Control Syst. Lett.*, 2025.

[13] M. Seidel, M. Maggio, and F. Allgöwer, "A Controller Synthesis Framework for Weakly-Hard Control Systems," in *Proc. 32nd IEEE Real-Time and Embedded Technology and Appl. Symp. (RTAS)*, 2026, pp. 41–54.

[14] M. Gäfvert, *Modelling the Furuta Pendulum*, ser. Technical Reports TFRT-7574. Dep. Autom. Control, Lund Inst. of Techn. (LTH), 1998.

[15] C. Mandrioli, S. Y. Shin, M. Maggio, D. Bianculli, and L. Briand, "Stress testing control loops in cyber-physical systems," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 2, 2023.

[16] C. M. Kirsch and A. Sokolova, "The logical execution time paradigm," in *Advances in Real-Time Systems*. Springer, 2012, pp. 103–120.

[17] L. Schenato, "To zero or to hold control inputs with lossy links?" *IEEE Trans. Automat. Control*, vol. 54, no. 5, pp. 1093–1099, 2009.

[18] N. Vreman, A. Cervin, and M. Maggio, "Stability and performance analysis of control systems subject to bursts of deadline misses," in *Proc. 33rd Euromicro Conf. Real-Time Syst. (ECRTS)*, 2021.

[19] R. I. Davis and L. Cucu-Grosjean, "A survey of probabilistic timing analysis techniques for real-time systems," *Leibniz Trans. Embedded Systems*, pp. 1–60, 2019.

[20] ——, "A survey of probabilistic schedulability analysis techniques for real-time systems," *Leibniz Trans. Embedded Systems*, vol. 6, no. 1, pp. 04:1–04:53, 2019.

[21] F. Marković, G. v. d. Brüggen, M. Günzel, J.-J. Chen, and B. B. Brandenburg, "A Distribution-Agnostic and Correlation-Aware Analysis of Periodic Tasks," in *Proc. 45th Real-Time Systems Symp. (RTSS)*, 2024.

[22] D. J. Antunes and H. Qu, "Frequency-Domain Analysis of Networked Control Systems Modeled by Markov Jump Linear Systems," *IEEE Trans. Control Netw. Syst.*, vol. 8, no. 2, pp. 906–916, 2021.

[23] N. Vreman and M. Maggio, "Stochastic Analysis of Control Systems Subject to Communication and Computation Faults," *ACM Trans. Embedded Computer Systems*, vol. 22, no. 5s, 2023.

[24] M. Hamdaoui and P. Ramanathan, "A dynamic priority assignment technique for streams with (m, k)-firm deadlines," *IEEE Trans. Computers*, vol. 44, no. 12, pp. 1443–1451, 1995.

[25] G. Bernat, A. Burns, and A. Liamosi, "Weakly hard real-time systems," *IEEE Trans. Computers*, vol. 50, no. 4, pp. 308–321, 2001.

[26] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Addison Wesley, 2006.

[27] N. Vreman, R. Pates, and M. Maggio, "WeaklyHard.jl: Scalable Analysis of Weakly-Hard Constraints," in *Proc. 28th Real-Time and Embedded Technology and Appl. Symp. (RTAS)*, 2022, pp. 228–240.

[28] N. Vreman, P. Pazzaglia, V. Magron, J. Wang, and M. Maggio, "Stability of Linear Systems Under Extended Weakly-Hard Constraints," *IEEE Control Syst. Lett.*, vol. 6, pp. 2900–2905, 2022.

[29] S. Linsenmayer and F. Allgöwer, "Stabilization of networked control systems with weakly hard real-time dropout description," in *Proc. IEEE 56th Conf. Decis. Control (CDC)*, 2017, pp. 4765–4770.

[30] S. Linsenmayer, M. Hertneck, and F. Allgöwer, "Linear weakly hard real-time control systems: Time- and event-triggered stabilization," *IEEE Trans. Automat. Control*, vol. 66, no. 4, pp. 1932–1939, 2021.

[31] M. Seidel, S. Lang, and F. Allgöwer, "On ℓ_2-performance of weakly-hard real-time control systems," *European J. Control*, vol. 80, p. 101056, 2024.

[32] K. J. Åström and R. Murray, *Feedback systems: an introduction for scientists and engineers*. Princeton university press, 2021.

[33] S. K. Ghosh, S. Dey, D. Goswami, D. Mueller-Gritschneder, and S. Chakraborty, "Design and validation of fault-tolerant embedded controllers," in *Proc. 2018 Design, Automation & Test in Europe Conf. & Exhibition*, 2018, pp. 1283–1288.

[34] P. Pazzaglia, "Performance-Driven Design for Control and Scheduling in Real-Time Systems," PhD thesis, Scuola Superiore Sant'Anna, 2020.

[35] G. C. Calafiore and M. C. Campi, "The scenario approach to robust control design," *IEEE Trans. Automat. Control*, vol. 51, no. 5, 2006.

[36] D. Fontanelli, L. Palopoli, and L. Abeni, "The continuous stream model of computation for real-time control," in *Proc. 34th IEEE Real-Time Systems Symp. (RTSS)*, 2013, pp. 150–159.

[37] M. Hertneck, S. Linsenmayer, and F. Allgöwer, "Stability Analysis for Nonlinear Weakly Hard Real-Time Control Systems," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 2594–2599, 2020.

[38] ——, "Efficient stability analysis approaches for nonlinear weakly-hard real-time control systems," *Automatica*, vol. 133, p. 109868, 2021.

[39] J. Löfberg, "YALMIP : a toolbox for modeling and optimization in MATLAB," in *Proc. Int. Conf. on Robot. and Automat.*, 2004.

[40] B. J. Josephrexon and M. Maggio, "Experimenting with networked control software subject to faults," in *Proc. Conf. on Decision and Control (CDC)*, 2022, pp. 1547–1552.

[41] A. Kiselev, A. Kuznietsov, and R. Leidhold, "Performance investigation of generalized predictive position control for a PMSM in view of reference trajectory tracking," in *Proc. IEEE Int. Conf. on Ind. Technol.*, 2018, pp. 481–485.

[42] K.-E. Arzén, A. Cervin, J. Eker, and L. Sha, "An introduction to control and scheduling co-design," in *Proc. 39th IEEE Conf. Decis. Control (CDC)*, vol. 5, 2000, pp. 4865–4870.

[43] Q. Zhu and A. Sangiovanni-Vincentelli, "Codesign Methodologies and Tools for Cyber–Physical Systems," *Proceedings of the IEEE*, vol. 106, no. 9, pp. 1484–1500, 2018.

[44] A. Cervin, B. Lincoln, J. Eker, K.-E. Årzén, and G. Buttazzo, "The jitter margin and its application in the design of real-time control systems," in *Proc. 10th Int. Conf. on Real-Time and Embedded Computing Systems and Appl.*, 2004.

[45] G. Buttazzo and A. Cervin, "Comparative assessment and evaluation of jitter control methods," in *Proc. 15th Int. Conf. on Real-Time and Network Systems (RTNS)*, 2007.

[46] A. Aminifar, P. Eles, Z. Peng, and A. Cervin, "Control-quality driven design of cyber-physical systems with robustness guarantees," in *Proc. Design, Automation & Test in Europe Conf. & Exhibition*, 2013.

[47] Y. Xu, K.-E. Årzén, A. Cervin, E. Bini, and B. Tanasa, "Exploiting job response-time information in the co-design of real-time control systems," in *Proc. 21st IEEE Int. Conf. Embedded and Real-Time Computing Systems and Appl.*, 2015, pp. 247–256.

[48] A. Cervin, P. Pazzaglia, M. Barzegaran, and R. Mahfouzi, "Using jittertime to analyze transient performance in adaptive and reconfigurable control systems," in *Proc. 24th IEEE Int. Conf. Emerging Technologies and Factory Automation (ETFA)*, 2019, pp. 1025–1032.

[49] A. Cervin, J. Eker, B. Bernhardsson, and K.-E. Årzén, "Feedback–feedforward scheduling of control tasks," *Real-Time Systems*, vol. 23, no. 1, pp. 25–53, 2002.

[50] G. Buttazzo, M. Velasco, and P. Marti, "Quality-of-control management in overloaded real-time systems," *IEEE Trans. Computers*, vol. 56, no. 2, pp. 253–266, 2007.

[51] T. Yoshimoto and T. Ushio, "Optimal arbitration of control tasks by job skipping in cyber-physical systems," in *Proc. IEEE/ACM 2nd Int. Conf. Cyber-Physical Systems*, 2011, pp. 55–64.

[52] Z. Wang, C. Huang, H. Kim, W. Li, and Q. Zhu, "Cross-layer adaptation with safety-assured proactive task job skipping," *ACM Trans. Embedded Computing Systems (TECS)*, vol. 20, no. 5s, pp. 1–25, 2021.

[53] X.-M. Zhang, Q.-L. Han, and X. Yu, "Survey on Recent Advances in Networked Control Systems," *IEEE Trans. Ind. Inform.*, vol. 12, no. 5, pp. 1740–1752, 2016.

[54] D. Fontanelli, L. Greco, and L. Palopoli, "Soft real-time scheduling for embedded control systems," *Automatica*, vol. 49, no. 8, 2013.

[55] P. Ramanathan, "Overload management in real-time control applications using (m, k)-firm guarantee," *IEEE Trans. Parallel and Distributed Systems*, vol. 10, no. 6, pp. 549–559, 1999.

[56] N. Jia, Y.-Q. Song, and R.-Z. Lin, "Analysis of networked control system with packet drops governed by (m,k)-firm constraint," *IFAC Proceedings Volumes*, vol. 38, no. 2, pp. 63–70, 2005.

[57] F. Flavia, J. Ning, S. YeQiong, and S.-L. Francoise, "Impact of a (m,k)-firm Data Dropouts Policy on the Quality of Control," in *Proc. 6th IEEE Int. Workshop on Factory Communication Systems*, 2006, pp. 353–359.

[58] A. Gujarati, M. Nasri, R. Majumdar, and B. Brandenburg, "From iteration to system failure: Characterizing the fitness of periodic weakly-hard systems," in *Proc. 31st Euromicro Conf. Real-Time Sys.*, 2019.

[59] Q. Zhu, W. Li, H. Kim, Y. Xiang, K. Wardega, Z. Wang, Y. Wang, H. Liang, C. Huang, J. Fan *et al.*, "Know the unknowns: Addressing disturbances and uncertainties in autonomous systems," in *Proc. 39th Int. Conf. Computer-Aided Design*, 2020, pp. 1–9.

[60] G. Frehse, A. Hamann, S. Quinton, and M. Woehrle, "Formal analysis of timing effects on closed-loop properties of control software," in *Proc. IEEE Real-Time Systems Symp. (RTSS)*, 2014, pp. 53–62.

[61] W. Geelen, D. Antunes, J. P. Voeten, R. R. Schiffelers, and W. Heemels, "The impact of deadline misses on the control performance of high-end motion control systems," *IEEE Trans. Ind. Electron.*, vol. 63, no. 2, pp. 1218–1229, 2015.

[62] R. Blind and F. Allgöwer, "Towards Networked Control Systems with guaranteed stability: Using weakly hard real-time constraints to model the loss process," in *Proc. 54th IEEE Conf. on Decision and Control (CDC)*, 2015, pp. 7510–7515.

[63] D. Goswami, R. Schneider, and S. Chakraborty, "Relaxing signal delay constraints in distributed embedded controllers," *IEEE Trans. Control Systems Technology*, vol. 22, no. 6, pp. 2337–2345, 2014.

[64] H. Liang, Z. Wang, D. Roy, S. Dey, S. Chakraborty, and Q. Zhu, "Security-driven codesign with weakly-hard constraints for real-time embedded systems," in *Proc. IEEE 37th Int. Conf. Computer Design (ICCD)*, 2019, pp. 217–226.

[65] P. Pazzaglia, L. Pannocchi, A. Biondi, and M. Di Natale, "Beyond the weakly hard model: Measuring the performance cost of deadline misses," in *Proc. 30th Euromicro Conf. Real-Time Syst. (ECRTS)*, 2018.

[66] H. Liang, Z. Wang, R. Jiao, and Q. Zhu, "Leveraging weakly-hard constraints for improving system fault tolerance with functional and timing guarantees," in *Proc. 39th Int. Conf. Comput.-Aided Des.*, 2020.

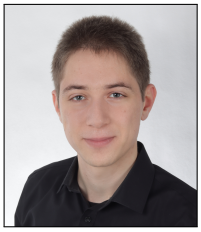
[67] C. Huang, W. Li, and Q. Zhu, "Formal verification of weakly-hard systems," in *Proc. 22nd ACM Int. Conf. Hybrid Systems: Computation and Control*, 2019, pp. 197–207.

[68] C. Huang, K.-C. Chang, C.-W. Lin, and Q. Zhu, "SAW: A tool for safety analysis of weakly-hard systems," in *Proc. Int. Conf. Computer Aided Verification*. Springer, 2020, pp. 543–555.

[69] K. Salamun, I. Pavić, H. Džapo, and I. Čuljak, "Weakly Hard Real-Time Model for Control Systems: A Survey," *Sensors*, vol. 23, no. 10, 2023.



Melanie Gallant is a PhD candidate at Bosch Corporate Research in Renningen, Germany, supervised by Prof. Martina Maggio at the Department of Computer Science, Saarland University. She obtained her Master's degree in Electrical Engineering and Information Technology at the Karlsruhe Institute of Technology in 2023. Her research focuses on stochastic and distributionally robust control methods for switched systems and on real-time control systems with uncertain timing effects.



Marc Seidel received the master's degree in engineering cybernetics from the University of Stuttgart, Germany, in 2021. He is currently a PhD student at the Institute for Systems Theory and Automatic Control at the University of Stuttgart, Germany. His research interests include real-time control systems, switched systems, and networked control systems, with particular focus on developing rigorous controller design methods with theoretic guarantees for cyber-physical systems subject to deadline misses.



Paolo Pazzaglia is a Research Engineer at Bosch Corporate Research, Renningen, Germany, since 2022. Earlier, he was a PostDoc Researcher at the Computer Science Department of Saarland University. He obtained his PhD in Emerging Digital Technologies (curriculum Embedded Systems) at Scuola Superiore Sant'Anna, Pisa, in 2020. His research merges control and real-time systems methods, targeting distributed and embedded control applications, with particular focus on guaranteeing joint timing and functional constraints.



Claudio Mandrioli is a Research Associate at the Software Verification and Validation group of the Interdisciplinary Centre for Security Reliability and Trust, part of the University of Luxembourg. His research combines methods from software engineering with control engineering models and tools, to enhance the verification and validation of cyber-physical systems, with emphasis on testing and specification languages.



Christoph Mark received the master's degree in Engineering Cybernetics from the University of Stuttgart, Germany, in 2017, and the Ph.D. degree in Control Engineering from the University of Kaiserslautern, Germany, in 2023. He is currently a Research Engineer at Bosch Corporate Research, Renningen, Germany. His research interests are centered around model predictive control, stochastic and distributionally robust optimization, optimal transport and uncertainty quantification.



Kevin Schmidt received the Ph.D. in engineering cybernetics from the University of Stuttgart, Germany, in 2020. His thesis focused on distributed parameter systems for optomechatronics. He was with the Cymer Center for Control Systems and Dynamics, UC San Diego, USA, in 2016. Since 2021 he is research engineer and project manager at Bosch Corporate Research, and lecturer for safety-critical control at the University of Stuttgart. His research interests are timing- & uncertainty-aware modeling, identification, and optimization of dynamic systems.



Frank Allgöwer studied Engineering Cybernetics and Applied Mathematics in Stuttgart and at the University of California, Los Angeles (UCLA), respectively, and received his Ph.D. degree from the University of Stuttgart in Germany. Since 1999 he is the Director of the Institute for Systems Theory and Automatic Control and professor at the University of Stuttgart. His research interests include networked control, cooperative control, predictive control, and nonlinear control with application to a wide range of fields including systems biology. For the years 2017–2020 Frank served as President of the International Federation of Automatic Control (IFAC) and for the years 2012-2020 as Vice President of the German Research Foundation DFG.



Martina Maggio is a Full Professor at the Computer Science Department of Saarland University, where she leads the Real-Time Systems and Control chair, and a Full Professor (20 % appointment) at the Department of Automatic Control, Lund University. Her research combines control theory, real-time systems, and software engineering to ensure the reliable operation of cyber-physical systems in the presence of timing failures such as deadline misses and sensor data misalignments.